C64uRemote for the M5StickC Plus2

Technical Documentation

Version 1.0

Contents

Overview

This build of C64uRemote runs on an **M5StickC Plus2** and drives a **Commodore 64 Ultimate** through its ReST API. Three building blocks were added compared to the first StickC version, all ported from the M5Stack Core edition:

- **NFC through a Unit RFID2** on the Grove port — read Wi-Fi cards and command cards, and write both
- **Wi-Fi configuration at runtime** — up to four networks in NVS instead of fixed values from `build_env.h`
- **Setup portal** — a private access point with a small web interface

Everything that needs a microSD card was left out: file browser, upload to the C64, card dump and restore. The stick has no card slot.

The card format is deliberately identical to the M5Dial and M5Stack Core. A card created on any of the devices works on all of them.

Hardware

Target platform

SoC	ESP32-PICO-V3-02, 240 MHz, dual core
Flash	8 MB (in package)
PSRAM	2 MB (in package)
Display	ST7789, 135 × 240, used in landscape (240 × 135)
Backlight	GPIO 27, via PWM
Power hold pin	GPIO 4 — must be HIGH, or the device switches itself off on battery
Input	button A (GPIO 37), button B (GPIO 39), power button through the PMU

The ESP32-PICO-V3-02 matters here: it differs from the PICO-D4 of the old M5StickC by its **integrated PSRAM** and a different package marking in the eFuse (`pkg_ver == 6`). That marking is exactly how M5GFX recognises the Plus2 — see *Build environment*.

Two separate I²C buses

This is the central wiring decision of this build:

Bus	Pins	Device	Address
`Wire`	SDA = G0, SCL = G26	Hat MiniJoyC (HAT port)	0x54
`Wire1`	SDA = G32, SCL = G33	Unit RFID2 / WS1850S (Grove port)	0x28

Both run in parallel without getting in each other's way. The MFRC522 driver receives its bus instance in the constructor:

```
MFRC522_I2C rfid(kRfidAddr, -1, &Wire1);
```

`Wire1.begin(32, 33, 100000UL)` has to run before `initRfid()` — `PCD_Init()` does **not** start the bus itself.

The MiniJoyC is driven without a library: `miniJoyBegin()` starts the bus and pings the address, `miniJoySetLedColor()` writes three bytes to register 0x40, and reading goes through `joyReadRegister()`. That saves a dependency and the ambiguity warnings from that library's source.

Detection at startup

```
Wire.begin(0, 26, 100000)      -> ping MiniJoyC  -> miniJoyReady
Wire1.begin(32, 33, 100000)    -> initRfid()     -> app.rfidReady
```

`initRfid()` first checks with an empty I²C transaction whether anything answers at 0x28, and only then calls `PCD_Init()`. The version register is merely logged and **not** evaluated: the WS1850S does not always report the same value there as a genuine MFRC522.

Build environment

PlatformIO

```
cp src/build_env.h.example src/build_env.h   # fill in
pio run -t upload
pio device monitor
```

In VS Code it is enough to open the project folder; the PlatformIO IDE creates `.vscode/c_cpp_properties.json` itself. `.vscode/extensions.json` and `.vscode/settings.json` ship with the project and suggest the PlatformIO extension when you open the folder.

Board settings — the dark-display trap

There is **no dedicated board entry** for the StickC Plus2 (as of espressif32 6.9). One therefore uses the M5StickC entry — but not unchanged.

The board manifest `m5stick-c.json` sets `-DARDUINO_M5Stick_C`. M5GFX uses that marker in `init_impl()` as its starting point:

```
#elif defined ( ARDUINO_M5STICK_C ) || defined ( ARDUINO_M5Stick_C )
      nvs_board = board_t::board_M5StickC;          // = 3
```

and passes it to `autodetect()`. There, the branch for the PICO-V3-02 reads:

```
else if (pkg_ver == 6)      // PICOV3_02 (StickCPlus2 / ATOM PSRAM)
{
  if (board == 0 || board == board_t::board_M5StickCPlus2)
```

With the value 3 the condition is false. The branch is skipped, **no panel is initialised at all**, and the power hold pin G4 is never asserted. The result is a dark screen with no error message. The marker is not a hint, it is a commitment.

`platformio.ini` corrects this in three places:

```
board_build.extra_flags =                ; drop -DARDUINO_M5Stick_C

build_flags =
    -DM5GFX_BOARD=board_t::board_M5StickCPlus2
    -DBOARD_HAS_PSRAM

build_unflags =
    -DARDUINO_M5Stick_C
```

`M5GFX_BOARD` is checked ahead of the whole `#elif` chain and therefore wins. In addition `setup()` sets:

```
cfg.fallback_board = m5::board_t::board_M5StickCPlus2;
```

To verify: the serial monitor must show `[Autodetect] M5StickCPlus2`.

M5GFX stores a detected board in NVS and reads it back on the next start **before** any compile-time marker. If a wrong value ends up there, only `pio run -t erase` helps — which also wipes the stored Wi-Fi credentials.

PSRAM

The PICO-V3-02 has 2 MB of PSRAM in the package, but the Arduino core only calls `psramInit()` when `BOARD_HAS_PSRAM` is defined. Without that marker `ps_malloc()` simply returns `NULL`:

```
void *ps_malloc(size_t size){
    if(!spiramDetected){ return NULL; }
    ...
}
```

Hence `-DBOARD_HAS_PSRAM` in the `build_flags`. The code does not rely on it regardless:

```
uint16_t* allocFrameBuffer(size_t bytes) {
  uint16_t* buffer = nullptr;
  if (psramFound()) buffer = static_cast<uint16_t*>(ps_malloc(bytes));
  if (buffer == nullptr) buffer = static_cast<uint16_t*>(malloc(bytes));
  return buffer;
}
```

Memory budget

Buffer	Size	Location
`M5Canvas` (full-screen sprite)	240 × 135 × 2 = 64.8 kB	internal
`plainLogoPixels`	64.8 kB	PSRAM, else internal
`boxedLogoPixels`	64.8 kB	PSRAM, else internal

While the **setup portal** is running, both logo caches are released: the access point and the web server need the internal heap. Three places belong together here and must be considered as a set when changing anything:

- `startPortal()` calls `releaseLogoCache()`
- `stopPortal()` calls `ensureLogoCache()`
- `render()` has a null-pointer branch for the home screen without a cache

If allocation fails, the device keeps running without logo effects and reports *WENIG SPEICHER* (low memory). An earlier revision looped forever at this point — a dark screen with no hint whatsoever.

Configuration (build_env.h)

```
#define C64U_WIFI_SSID       "MyNetwork"
#define C64U_WIFI_PASSWORD   "MyWifiPassword"
#define C64U_TARGET_HOST     "192.168.0.64"
#define C64U_TARGET_PASSWORD ""
```

These values are **initial values only**. `loadNetConfig()` reads NVS first; only if nothing was ever stored there (the key `wcount` is missing) is the entry from `build_env.h` adopted as the first profile. The fields may therefore be left empty.

Wi-Fi subsystem

Runtime configuration instead of compile time

```
struct WifiProfile { String ssid; String pass; };
WifiProfile gWifiProfiles[kWifiProfileMax];   // kWifiProfileMax = 4
size_t      gWifiCount = 0;
size_t      gWifiTry   = 0;
String      gTargetHost, gTargetPass;
```

`wifiAddProfile()` moves a network to the front (most recently used first); when the list is full, the last entry drops out. `beginWiFi()` takes `gWifiProfiles[gWifiTry]` and then advances, so `serviceWiFi()` tries a different network every 10 s until one answers.

At startup `setup()` calls `wifiPickBestProfile()` once: a blocking scan that moves the known network with the strongest signal to the front. That only pays off with more than one stored network and is skipped otherwise.

NVS layout

Two namespaces, so that a *Factory Reset* of the display settings does not take the credentials with it:

`c64uremote` — settings

Key	Content
`anim_on`, `fx_mode`, `anim_spd`, `fx_time`, `st_time`, `bright`	display
`joy_ovl`, `joy_led`, `joy_led_br`, `joy_thr`	MiniJoyC
`auto_nfc`	background polling interval
`card_cnf`	prompt time of a PowerOff command card, in seco

`c64unet` — network

Key	Content
`host`, `hostpw`	address and password of the c64u
`wcount`	number of stored networks (0–4)
`s0`…`s3`	SSIDs
`p0`…`p3`	passwords

`wcount` doubles as the marker “something has been saved here before”.

Card format

Wi-Fi cards use the scheme from the Wi-Fi QR code:

```
WIFI:S:MyNetwork;T:WPA;P:secret;;
```

`parseWifiText()` evaluates the fields `S:` and `P:`; `T:` is ignored (the type follows from the password). Special characters are escaped with a backslash; the parser works character by character with an escape state, so `;` and `:` inside a password work. `wifiCardText()` is the counterpart for writing.

`textLooksLikeWifi()` only checks the prefix `WIFI:` — that is how background polling spots a Wi-Fi card without parsing it fully.

Setup portal

SSID	`C64uRemote-Setup`
Password	`c64ultimate`
Address	`192.168.4.1`
Timeout	5 min without use
Grace period after saving	2.5 s, so the response page still arrives

A `DNSServer` answers every name with its own address and `onNotFound` redirects to the start page — together that makes a captive portal which opens by itself on most devices.

Password fields left empty keep the previous value. If the SSID is already stored, its password is carried over.

NFC subsystem

Polling strategy

`serviceRfid()` has two modes:

On the card screens (`NfcRead`, `NfcWrite`, `WifiCard`) it asks every 250 ms with `cardPresent()` — full reader timeout.

On the home screen background polling runs at the *Auto-NFC* interval (1.5 s / 0.7 s / 0.3 s) and uses `cardPresentQuick()`.

The difference matters for responsiveness. With no card present, the MFRC522 waits after the REQA command until its internal timer expires. `PCD_Init()` configures 0x03E8 = 1000 steps of 25 µs each — **25 ms** during which the main loop stalls. A card answers far sooner (frame delay time at 106 kbit/s is about 86 µs), so `cardPresentQuick()` lowers the timeout to 80 steps (2 ms) for the probe alone and restores the original value immediately afterwards — before card selection. Authentication, reading and writing therefore still run with the full timeout.

A card left on the reader is not executed over and over thanks to a 2.5 s lock (`kRfidRepeatMs`). The exception is the prompt of a PowerOff card: that may be confirmed by presenting the card again at any time.

Card types

Family	Access
NTAG213/215/216, MIFARE Ultralight	4 bytes per page, NDEF TLV from page 4, no key
MIFARE Classic 1K/4K/Mini	16 bytes per block, NDEF TLV from block 4, sector trailers skipped

For Classic cards authentication is attempted first with the NDEF key `D3F7D3F7D3F7`, then with the factory key `FFFFFFFFFFFF`. The last successful key is remembered per card, otherwise every block costs a pointless failed attempt plus a reselect.

Sector trailers and the MAD are never written. A card cannot be bricked by this program; an unformatted Classic card may afterwards only be readable on this device.

After a failed authentication the card sits in HALT state and only answers WUPA. `reselectCard()` therefore wakes it deliberately with `PICC_WakeupA()` and selects it again — a plain `PICC_IsNewCardPresent()` (REQA) would no longer find it.

Data format

What gets written is always a single NDEF text record, UTF-8, language code `en`:

```
TLV     : 03 <len> ... FE
Record  : D1 01 <plen> 54 | 02 'e' 'n' | <text>
          D1 = MB|ME|SR|TNF=1 (Well Known), 54 = 'T'
```

The buffer is padded to a multiple of 16 so that whole blocks are written. After writing, `writeCardText()` reads the card back and compares the text.

When reading, the old raw format with the `C64UPATH` magic is also recognised, so previously written cards keep working.

The parser is deliberately tolerant: some writers store a wrong payload length — the TeensyROM writes a constant 0x10 there even though the TLV length is correct. For the last record (ME flag) the TLV length therefore takes precedence.

Command cards

```
CMD:RESET
CMD:REBOOT
CMD:MENU
CMD:POWEROFF=0      switch off immediately
CMD:POWEROFF=8      ask first, 8 s to confirm
CMD:POWEROFF        ask first, using the device setting "NFC-Cmd PowOff"
CMD:CPU=10          set the CPU to 10 MHz
CMD:JOY             toggle the joystick ports (Normal <-> Swapped)
CMD:JOY=SWAPPED     set the ports fixed; also NORMAL, WASD1, WASD2
```

`parseCardCommand()` is insensitive to case and spaces. Without an argument, `POWEROFF` uses the device setting *NFC-Cmd PowOff* (3/5/8/15 s); `0` means “no prompt”.

The prompt is tracked in `app.cardPowerOffPending` together with the UID and an expiry time. It is confirmed by presenting **the same** card again or by a button press (`activateCurrent()` intercepts that ahead of everything else).

What the stick cannot do

`processCard()` recognises path cards but does not execute them:

```
app.rfidHint = "Programmkarte - dafuer fehlt die SD";
setModal("BRAUCHT SD-KARTE", ...);
```

Launching a program would require reading the file from a local microSD and streaming it to the C64. Both exist only in the Core and Dial editions.

Software architecture

State model

`ScreenMode` drives both rendering and input:

State	Content
`Home`	logo / effects, background polling of the reader
`Menu`	main menu
`CpuMenu`	clock speeds
`Status`	connection overview
`DisplaySettings`	settings list
`NfcMenu`	NFC / RFID submenu
`NfcRead`	present a card → execute its content
`CmdPick`	pick a command for a card
`NfcWrite`	present a card → write text
`WifiMenu`	Wi-Fi submenu
`WifiCard`	present a card → read credentials
`WifiSaved`	stored networks, optionally in delete mode
`WifiPortal`	setup access point running

Navigation

So that a new screen does not drag three more `switch` branches along with it, all input goes through three functions:

```
void moveSelection(int delta);     // scroll
void activateCurrent(uint32_t);    // select / execute
void goBack(uint32_t);             // one level back
```

`listSelection()` supplies the pointer and length of the current screen’s list. Buttons and the MiniJoyC only call those three; the home screen’s special cases (double click = reboot, joystick up = reset) live directly in the button handling.

`activateCurrent()` intercepts an open PowerOff prompt ahead of everything else.

Rendering

Drawing always goes into an `M5Canvas` and is pushed to the display in one piece — hence no flicker. The frame rate is about 30 fps (`kFrameMs = 33`).

On the home screen a redraw only happens when something changed (`app.home.frameDirty`) or an effect is running. The effects read from `plainLogoPixels`, the pre-rendered logo; `boxedLogoPixels` holds the same graphic with a cleared inner area.

ReST interface to the C64 Ultimate

The base is `http://<host>` with a 2.5 s timeout. If a password is configured it travels as the `X-Password` header.

Purpose	Method and path
Reachability / version	`GET /v1/version`
Reset	`PUT /v1/machine:reset`
Reboot	`PUT /v1/machine:reboot`
Power off	`PUT /v1/machine:poweroff`
Ultimate menu	`PUT /v1/machine:menu_button`
Configuration tree	`GET /v1/configs`
Read CPU value	`GET /v1/configs/<category>/<item>`
Set CPU value	`PUT /v1/configs/<category>/<item>?value=<value>`
Read/set joystick ports	the same `/v1/configs` paths

`resolveCpuPath()` looks the CPU setting up in the configuration tree once — the Ultimate firmware names it differently depending on the version. If that fails, a built-in fallback list of the usual clock speeds is used.

There is no `machine:` command for the joystick ports. They are a configuration item as well - in testing *Joystick Swapper* in *U64 Specific Settings*, with the values `Normal`, `Swapped`, `WASD Port 2`, `WASD Port 1`. `resolveJoyPath()` looks it up the same way (first *U64 Specific Settings*, otherwise every category until an item name contains “Joystick”). `joyTokenFromValue()` and `joyValueFromToken()` convert between the device value and the card token (`WASD Port 1 <-> WASD1`). One caveat when reading the source: everything else named `joy` belongs to the MiniJoyC on the HAT port.

`refreshConnectionStatus()` probes at most every 15 s, first without and then with the password. The four states of the status LED follow from that.

Refused connections

The HTTP server of the Ultimate firmware accepts only one connection at a time and refuses further ones with a TCP RST; `HTTPClient` reports this as *connection refused*. It was observed with only a single device on the network as well - so it happens sporadically and is neither a radio nor an address problem.

The actual call therefore moved into `sendApiRequestOnce()`. `sendApiRequest()` is only a wrapper around it: if the transport fails (`httpCode <= 0`), a second attempt follows after `kApiRetryDelayMs` (250 ms). Retrying happens **only** on transport errors - nothing reached the c64u then, so a command cannot be doubled. HTTP error statuses (4xx, 5xx) are passed through unchanged, and the streaming upload in `uploadFile()` has its own path and stays untouched.

On top of that `refreshConnectionStatus()` saves a request: without a stored password the second query would be byte-identical to the first, because the `X-Password` header is only set when there is one. That halves the base load on the c64u.

Reconnecting with several networks

`beginWiFi()` moves on to the next stored profile after every attempt. Without a countermeasure that means: with two networks stored of which only one is reachable, every other reconnect after a dropout hits the dead one and costs a full retry cycle (`kWiFiRetryMs`, 10 s).

`serviceWiFi()` therefore remembers the profile the connection came up on, via `wifiProfileIndex(WiFi.SSID())`, and makes it the next attempt; `gWifiNoted` keeps this to once per connection. After a dropout the first attempt goes back to the working network, and the dead one is only tried if the good one is really gone.

Input logic

Buttons

Button	Home screen	Lists
A	reset; twice within 300 ms = reboot	`goBack()`
B	Ultimate menu	`moveSelection(+1)`
Power (short)	open the menu	`activateCurrent()`

The single click on A is executed deliberately late: the reset only runs once 300 ms have passed without a second press. There is no other way to put reset and reboot on the same button.

Blocking calls

Two places deliberately stall the main loop:

- **HTTP requests** for up to 2.5 s
- **Waiting for the Wi-Fi connection** after a Wi-Fi card for up to 8 s (`kWifiCardConnectMs`), with the message *VERBINDE…* drawn beforehand

In both cases a short stall is the more honest option: the user should see the result, not a menu that stays operable while nothing is settled underneath.

PowerOff safeguards

Switching off is guarded in three places:

1. The menu entry and joystick-down always ask — a second press within 2 s (`kPowerOffConfirmMs`)
2. Command cards with a prompt: present again or press a button within 3–15 s
3. `CMD:POWEROFF=0` switches off without asking — whoever creates such a card means it

Persistence

What	Where
Display and input settings	NVS `c64uremote`
Wi-Fi profiles, host, host password	NVS `c64unet`
Detected board	NVS, managed by M5GFX itself

Factory Reset in the settings only affects `c64uremote`. Network data is cleared under *WLAN → Alle loeschen*, everything at once with `pio run -t erase`.

Project layout

```
M5StickC_Plus2_C64uRemote/
├── platformio.ini            board, libraries, partition, upload
├── README.md
├── LICENSE                   MIT - Karl Prosser, Martin Oswald
├── .vscode/                  recommended extensions, editor settings
├── docs-src/                 markdown sources of the manuals
├── doc/                      finished manuals (PDF)
├── src/                      German edition
│   ├── main.cpp              entire firmware
│   ├── build_env.h           credentials (do not version)
│   ├── build_env.h.example   template
│   └── 1MHz_logo_rgb565.h    logo as an RGB565 array
└── src-en/
    └── main.cpp              English edition, identical code
```

Dependencies: M5Unified, M5GFX, ArduinoJson and the I²C driver `kkloesener/MFRC522_I2C`. There is deliberately no library for the MiniJoyC — the four accesses that are needed sit directly in the source.

Troubleshooting

The screen stays dark. Check `platformio.ini` against the *Board settings* chapter. The serial monitor must show `[Autodetect] M5StickCPlus2`. If that does not help: `pio run -t erase`, then flash again.

Message *WENIG SPEICHER*. The logo caches did not fit. Check that `-DBOARD_HAS_PSRAM` is set; otherwise the monitor shows `logo cache alloc failed` along with the free heap.

Everything under *NFC / RFID* says *kein NFC*. The reader was not found at startup. Check the Grove plug; the monitor shows `RFID2 nicht gefunden`. The reader is only looked for once at startup, so restart after plugging it in.

Cards are read but not written. On MIFARE Classic the data blocks may be secured with a foreign key. This program only tries the NDEF and the factory key; a dictionary attack lives deliberately only in the Core edition.

The setup portal does not open. Some phones drop a Wi-Fi network without internet access on their own. Choose “stay connected anyway” in the Wi-Fi settings and enter `192.168.4.1` in the browser by hand.

Sources

- Original project: Karl Prosser (@klumsy), https://github.com/ReadyOS-C64/C64uRemote
- Ultimate firmware and ReST API: https://1541u-documentation.readthedocs.io
- M5Unified / M5GFX: https://github.com/m5stack/M5Unified
- MFRC522 I²C driver: https://github.com/kkloesener/MFRC522_I2C
- This edition: Martin Oswald (@mad), https://1MHz.de

License

C64uRemote is released under the **MIT License**. The full text is in the file `LICENSE` in the project root.

It originates from **C64uRemote by Karl Prosser (@klumsy)**, https://github.com/ReadyOS-C64/C64uRemote, which he published under the MIT License. This version is a derivative work and is released under the same terms:

The MIT License allows you to use, modify and redistribute the software, including commercially. The only condition: **the copyright notice and the license text must be kept** and included with every copy. There is no warranty and no liability.

The libraries used carry their own licenses: M5Unified and M5GFX (MIT, © M5Stack), ArduinoJson (MIT, © Benoit Blanchon) and MFRC522_I2C (https://github.com/kkloesener/MFRC522_I2C). PlatformIO fetches them at build time.

How this was made

The port, the extensions and these manuals were written with the help of Claude (Anthropic). Concept, idea, hardware decisions and every test on real devices: Martin Oswald (@mad).